系统设计说明书

分布式计算任务安全分发与结果审计平台

项目 13 ｜ 文档编号 P13-SD-001 ｜ 版本 1.0 ｜ 日期 2026 年 11 月 11 日

班级：_____ 姓名：_____ 学号：_____

1 系统概述

本系统面向课程实验中的分布式计算场景，由 Python 服务端与 Windows C 客户端组成。服务端把整数区间拆分为子任务，分配给多个独立节点，收集带完整性证明的结果，通过严格多数表决形成可信结果，并保存任务、信誉和安全事件记录。系统提供求和、计数、最小值和最大值四类运算。

设计目标是把任务分发、节点认证、结果一致性校验和审计串联为可演示的完整流程。系统信任服务端和部署阶段发放的密钥；结果可靠性依赖参与表决节点中存在诚实多数。Merkle 证明校验步骤值与总结果的一致性，不能单独证明节点执行过程真实或计算正确。

1.1 功能范围

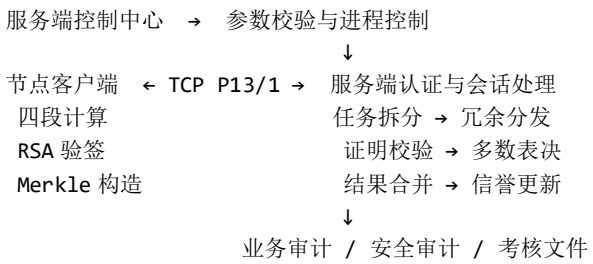
功能	设计与处理方式
任务管理	设置运算、总区间、子任务数和冗余节点数；逐轮推进子任务。
节点接入	校验许可证、时间戳、随机数、程序哈希和 HMAC 身份标签。
安全分发	建立临时会话密钥，对业务消息加密并认证；任务附 RSA 签名。
计算验证	客户端四段计算，提交步骤值与 Merkle 根；服务端验证并投票。
信誉与审计	按结果更新分值、识别累计错误节点，记录业务和安全事件。
可视化	服务端控制中心展示运行、节点、日志、审计、考核和自检信息。

1.2 部署与默认配置

默认监听地址为 127.0.0.1:8888，属于同机多进程演示。默认计算区间为 1 至 1000，拆分为 5 个子任务，每轮配置 10 个节点，线程池上限为 64。每轮至少 3 个节点，每个子任务至少包含 4 个整数。跨机器运行需同时调整客户端与服务端地址及部署配置，本版本默认配置不直接实现远程接入。

2 总体架构与模块职责

系统按界面与控制、业务、安全、网络和持久化分工。客户端通过 TCP 连接服务端，认证后以 SECURE 帧承载业务动作。服务端以 select 等待监听套接字就绪，把已接受连接提交给线程池；每个工作线程处理一个节点会话，共享任务状态由可重入锁保护。



模块	主要文件	职责
网络与协议	server.py / protocol.py	监听、并发接入、分帧、状态码和 JSON 编解码。
认证与会话	credential_store.py / secure_channel.py	许可证、程序哈希和 HMAC 认证，会话密钥及抗重放。
任务业务	task_service.py / task_state.py	领取任务、提交验证、轮次推进、共享状态加锁。
计算与验证	task_planner.py / operations.py / merkle.py	区间划分、运算及合并、四步骤证明校验。
签名与密钥	signature.py / rsa_keys.py	RSA-2048 私钥生成、SHA-256 任务签名。
信誉与记录	scheduler.py / audit.py / security_audit.py	资格筛选、业务与安全事件分开记录。
界面与评估	gui.py / process_manager.py / evaluation.py	启动控制、状态展示和指标计算。
客户端	client_logic.c 及 network、protocol、security 等模块	认证、验签、计算、结果提交与在线等待。

2.1 并发与一致性

任务领取时在锁内检查轮次、已分配节点集合和名额；提交时在锁内校验当前任务与重复结果。结果按节点 ID 保存，每轮同一节点最多计入一次。达到配置节点数才进行表决。未成功提交即断开时释放已分配名额，供其他节点领取。

3 任务计算与结果可信机制

3.1 区间拆分与运算

设总区间为 [A,B]，元素数 N=B−A+1，子任务数为 K，轮次 r 从 1 开始。子区间起点为 A+⌊N(r−1)/K⌋，终点为 A+⌊Nr/K⌋−1。该划分覆盖整个区间且互不重叠；不能整除时长度最多相差一个元素。客户端再以相同原则把领取区间拆为四段。

运算标识	子任务计算	总结果合并
SUM_RANGE	区间内各整数求和	对子任务结果求和
COUNT_RANGE	区间内整数个数	对子任务计数求和
MIN_RANGE	区间最小整数	取子任务最小值的最小值
MAX_RANGE	区间最大整数	取子任务最大值的最大值

3.2 任务水印与完整性

任务 ID 形如 task-会话片段-三位轮次号。服务端把 task_id、operation、start、end 以竖线连接，使用 RSA-2048 与 SHA-256/PKCS#1 v1.5 签名。客户端用部署公钥验签后执行。任务 ID 的随机片段用于区分运行批次，不等同于硬件可信执行证明。

每个步骤结果的十进制文本经过 SHA-256 得到叶子哈希，相邻叶子的十六进制哈希文本拼接后再次哈希，最终得到根。提交包含四个步骤值、根和合并值。服务端重新合并步骤并重建根；若只修改总值或根，证明不匹配。攻击者若同时构造自洽错误步骤和根，仍需依靠冗余表决识别。

3.3 表决与信誉

严格多数门槛为 ⌊已收集结果数/2⌋+1，当前流程收齐每轮配置数量后投票。存在严格多数时选其值并累计整合结果。无严格多数时保存失败审计，finish_task 返回失败；当前调用方仍可能推进轮次，因此不能把轮次结束等同于任务成功。

节点初始信誉为 60，正确一次加 5，错误一次减 15，限制在 0 至 100。错误次数累计达到 5 次时加入恶意节点集合；正确轮次不清空既有错误计数。信誉至少 60 且未被识别为恶意的节点进入重要任务资格名单。当前普通领取路径未引用该名单，资格筛选与实际强制拒发之间仍需补齐。

4 安全与数据设计

4.1 认证和安全会话

部署阶段生成独立节点身份密钥、许可证和程序 SHA-256 白名单。登录请求携带节点 ID、许可证 ID、Unix 秒时间戳、16 字节客户端随机数、程序哈希及 HMAC-SHA256 标签；persistent 标志也纳入认证文本。服务端验证许可证归属、有效期、正负 60 秒时间窗、随机数重复、白名单及身份标签。

认证成功后，服务端返回会话 ID、服务端随机数和身份确认标签。双方以身份密钥对 SESSION 前缀、双方随机数和会话 ID 进行 HMAC 派生。业务消息使用 HMAC 派生密钥流与明文异或，外加覆盖方向、会话、序号、nonce 和密文的认证标签。双向序号各自从 1 开始严格递增。该方案属于本项目自定义安全通道，不应当表述为 TLS 或标准 AES 加密。

4.2 数据对象

对象	关键字段	存储方式
任务状态	会话片段、轮次、已分配节点、结果、证明、可信值	内存；RLock 保护
节点信誉	reputation_scores、wrong_rounds、malicious_nodes	内存；审计留存快照
业务审计	任务 ID、轮次、范围、节点结果、证明、信誉、考核	data/audit.jsonl 逐行追加
安全事件	事件类型、节点、地址、详情与时间	data/security_audit.jsonl
考核及自检	误差、安全开销、恶意比例；自检用例结果	evaluation.json / security_tests.json
密钥与凭据	服务端私钥、节点身份密钥、许可证、公钥	受控部署文件；不放入报告正文

4.3 异常与限制

错误类型包括认证失败、身份不匹配、超长帧、证明无效、重复提交、网络超时及内部错误。认证前返回 ERROR，建立会话后返回携带 ERROR 动作的 SECURE 帧。服务端会话处理的套接字超时实际为 8 秒；配置中的 30 秒会话超时尚未形成独立强制过期流程。

任务与信誉状态未使用数据库，服务端重启后内存状态重置。服务端等待全部配置结果，尚缺少业务级任务截止时间和自动补位策略。TEE 远程证明未实现。C 客户端使用 long 存储数值，在 Windows 上通常为 32 位，输入及中间结果需控制在可表示范围内。

5 部署使用与当前交付状态

5.1 环境和操作流程

运行环境为 Windows、Python 3、支持 C11 和 Windows 系统库的 C 编译器。历史环境记录列出了 Python 3.14.6 与 GCC 4.9.2。本次文档复核使用配套 Python 3.12.14，仅验证明确列出的模块与算法，不代替历史机器的客户端构建验证。

操作顺序：先完成客户端构建及凭据签发，启动服务端控制中心并进入等待状态；手动打开使用不同节点 ID 的客户端，认证并保持在线；确认可用节点数量后选择运算、输入参数并开始任务；观察分段进度、表决结果及审计；任务完成后客户端可继续心跳等待，用户主动断开时发送 LOGOUT。

工程根目录下的主要入口
Python 服务端控制中心：`python server/gui.py`
凭据与白名单刷新：`python server/provision.py`
安全自检：`python server/security_tests.py`
控制台客户端：`client/client.exe node-01 normal stay`
重新编译客户端后应重新执行凭据签发脚本更新程序哈希白名单。首次签发和安全自检会写入项目数据文件，需在准备正式答辩演示时执行；本次复核没有覆盖原有凭据和历史审计。

5.2 需要完成的工程项

项目	本次核对结果	完成条件
客户端网络接口	network.h 声明的 connect_retry、send_bytes、receive_exact 在当前 network.c 中缺少实现	补齐定义并重新构建两个客户端，再进行 TCP 分段联调
启动任务配置	README 提到的.vscode/tasks.json 在当前目录未找到	补齐任务文件或改用已核验的手动启动流程
重要任务约束	可生成资格名单，但普通任务领取未强制使用	在任务分配入口落实资格判定并添加验证
表决失败处理	无多数后调用链仍可推进轮次	以表决成功为推进条件或增加失败重试状态

5.3 设计依据

主要依据：README.md、PROTOCOL.md、ENVIRONMENT.md；server 下的 config、server、task_service、task_state、task_planner、operations、credential_store、secure_channel、signature、merkle、scheduler、audit、evaluation；client 下的 client_logic、network、protocol、merkle。本文描述实现机制和明确的现存缺口，实际测试结论见配套测试验证报告。